

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA0910

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I B. Priyanka / 192421206 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. Priyanka

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

```
import java.util.Scanner;

class Book {

    private int bookId;

    private String title;

    private String author;

    private double price;

    private boolean issued;

    Book(int bookId, String title, String author, double price) {

        this.bookId = bookId;

        this.title = title;

        this.author = author;

        this.price = price;

        this.issued = false;

    }

    public int getBookId() {

        return bookId;

    }

    public void issueBook() {

        if (!issued) {

            issued = true;

            System.out.println("Book issued successfully.");

        } else {

            System.out.println("Book is already issued.");

        }

    }

}
```

```

    }

}

public void returnBook() {

    if (issued) {

        issued = false;

        System.out.println("Book returned successfully.");

    } else {

        System.out.println("Book was not issued.");

    }

}

public void display() {

    System.out.println("-----");

    System.out.println("Book ID : " + bookId);

    System.out.println("Title : " + title);

    System.out.println("Author : " + author);

    System.out.println("Price : " + price);

    System.out.println("Status : " + (issued ? "Issued" : "Available"));

    System.out.println("-----");

}

}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Book[] books = new Book[10];

        int count = 0;

        while (true) {

            System.out.println("\n===== Library Book Management =====");

            System.out.println("1. Add Book");

            System.out.println("2. Display Books");

```

```
System.out.println("3. Issue Book");

System.out.println("4. Return Book");

System.out.println("5. Search Book");

System.out.println("6. Exit");

System.out.print("Enter your choice: ");

int choice = sc.nextInt();

switch (choice) {

    case 1:

        if (count == books.length) {

            System.out.println("Library is Full!");

            break;

        }

        System.out.print("Enter Book ID: ");

        int id = sc.nextInt();

        sc.nextLine();

        System.out.print("Enter Title: ");

        String title = sc.nextLine();

        System.out.print("Enter Author: ");

        String author = sc.nextLine();

        System.out.print("Enter Price: ");

        double price = sc.nextDouble();

        books[count] = new Book(id, title, author, price);

        count++;

        System.out.println("Book Added Successfully.");

        break;

    case 2:

        if (count == 0) {

            System.out.println("No books available.");

        } else {
```

```
        for (int i = 0; i < count; i++) {  
            books[i].display();  
        }  
    }  
  
    break;
```

case 3:

```
    System.out.print("Enter Book ID to Issue: ");  
  
    id = sc.nextInt();  
  
    boolean found = false;  
  
    for (int i = 0; i < count; i++) {  
        if (books[i].getBookId() == id) {  
            books[i].issueBook();  
            found = true;  
            break;  
        }  
    }  
  
    if (!found) {  
        System.out.println("Book not found.");  
    }  
  
    break;
```

case 4:

```
    System.out.print("Enter Book ID to Return: ");  
  
    id = sc.nextInt();  
  
    found = false;  
  
    for (int i = 0; i < count; i++) {  
        if (books[i].getBookId() == id) {  
            books[i].returnBook();  
            found = true;  
            break;  
        }  
    }  
  
    break;
```

```

        }

    }

    if (!found) {

        System.out.println("Book not found.");

    }

    break;

case 5:

    System.out.print("Enter Book ID to Search: ");

    id = sc.nextInt();

    found = false;

    for (int i = 0; i < count; i++) {

        if (books[i].getBookId() == id) {

            books[i].display();

            found = true;

            break;

        }

    }

    if (!found) {

        System.out.println("Book not found.");

    }

    break;

case 6:

    System.out.println("Thank You!");

    sc.close();

    System.exit(0);

default:

    System.out.println("Invalid Choice.");

}

}

```

```

}

}

```

```

Main.java
1 import java.util.Scanner;
2
3 class Book {
4     private int bookId;
5     private String title;
6     private String author;
7     private double price;
8     private boolean issued;
9     Book(int bookId, String title, String author, double price) {
10         this.bookId = bookId;
11         this.title = title;
12         this.author = author;
13         this.price = price;
14         this.issued = false;
15     }
16     public int getBookId() {
17         return bookId;
18     }
19     public void issueBook() {
20         if (!issued) {
21             issued = true;
22             System.out.println("Book issued successfully.");
23         } else {
24             System.out.println("Book is already issued.");
25         }
26     }
27     public void returnBook() {
28         if (issued) {
29             issued = false;
30             System.out.println("Book returned successfully.");
31         } else {

```

```

Output
===== Library Book Management =====
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
Enter your choice: 1
Enter Book ID: 201
Enter Title: JAVA
Enter Author: MAGESH
Enter Price: 500
Book Added Successfully.

===== Library Book Management =====
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
Enter your choice: 2
-----
Book ID : 201
Title   : JAVA
Author  : MAGESH
Price   : 500.0
Status  : Available
-----

```

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

```

class Employee {

    protected int empId;

    protected String name;

    Employee(int empId, String name) {

        this.empId = empId;

        this.name = name;

    }

    double calculateSalary() {

        return 0;

    }

    void displayDetails() {

        System.out.println("Employee ID : " + empId);

        System.out.println("Employee Name : " + name);

        System.out.println("Salary      : " + calculateSalary());

```

```

    }

}

class PermanentEmployee extends Employee {

    double basicSalary;

    double hra;

    double da;

    PermanentEmployee(int empId, String name, double basicSalary, double hra, double da) {

        super(empId, name);

        this.basicSalary = basicSalary;

        this.hra = hra;

        this.da = da;

    }

    @Override

    double calculateSalary() {

        return basicSalary + hra + da;

    }

}

class ContractEmployee extends Employee {

    double hourlyRate;

    int hoursWorked;

    ContractEmployee(int empId, String name, double hourlyRate, int hoursWorked) {

        super(empId, name);

        this.hourlyRate = hourlyRate;

        this.hoursWorked = hoursWorked;

    }

    @Override

    double calculateSalary() {

        return hourlyRate * hoursWorked;

    }

}

```



```

    }

    public class Main {

        public static void main(String[] args) {

            Employee e1 = new PermanentEmployee(101, "John", 30000, 5000, 3000);

            Employee e2 = new ContractEmployee(102, "David", 500, 160);

            System.out.println("===== Permanent Employee =====");

            e1.displayDetails();

            System.out.println();

            System.out.println("===== Contract Employee =====");

            e2.displayDetails();

        }

    }
}

```

The screenshot shows an IDE with a file named 'Main.java'. The code defines an abstract class 'Employee' with attributes 'empId' and 'name', and methods 'calculateSalary()' and 'displayDetails()'. It has two subclasses: 'PermanentEmployee' and 'ContractEmployee'. The 'main' method in the 'Main' class creates instances of these two classes and calls 'displayDetails()' on them. The output window on the right shows the results of the program execution, displaying details for both employees and a success message.

```

Main.java
1- class Employee {
2-     protected int empId;
3-     protected String name;
4-     Employee(int empId, String name) {
5-         this.empId = empId;
6-         this.name = name;
7-     }
8-     double calculateSalary() {
9-         return 0;
10-    }
11-    void displayDetails() {
12-        System.out.println("Employee ID : " + empId);
13-        System.out.println("Employee Name : " + name);
14-        System.out.println("Salary : " + calculateSalary());
15-    }
16- }
17- class PermanentEmployee extends Employee {
18-     double basicSalary;
19-     double hra;
20-     double da;
21-     PermanentEmployee(int empId, String name, double basicSalary, double hra, double da) {
22-         super(empId, name);
23-         this.basicSalary = basicSalary;
24-         this.hra = hra;
25-         this.da = da;
26-     }
27-     @Override
28-     double calculateSalary() {
29-         return basicSalary + hra + da;
30-     }
31- }
32- class ContractEmployee extends Employee {
33-     double hourlyRate;
34-     int hoursWorked;
35-     ContractEmployee(int empId, String name, double hourlyRate, int hoursWorked) {

```

```

Output
===== Permanent Employee =====
Employee ID : 101
Employee Name : John
Salary : 38000.0

===== Contract Employee =====
Employee ID : 102
Employee Name : David
Salary : 80000.0

=== Code Execution Successful ===

```

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

```

class Vehicle {

    double calculateRent(int days) {

        return 0;
    }
}

```

```
}

void displayDetails(int days) {

    System.out.println("Rental Days : " + days);

    System.out.println("Total Rent : $" + calculateRent(days));

}

}

class Car extends Vehicle {

    @Override

    double calculateRent(int days) {

        return days * 1500;

    }

}

class Bike extends Vehicle {

    @Override

    double calculateRent(int days) {

        return days * 500;

    }

}

class Bus extends Vehicle {

    @Override

    double calculateRent(int days) {

        return days * 3000;

    }

}

public class Main {

    public static void main(String[] args) {

        Vehicle[] vehicles = {

            new Car(),

            new Bike(),
```

```

        new Bus()

    };

    String[] names = {

        "Car",

        "Bike",

        "Bus"

    };

    int days = 5;

    System.out.println("===== Vehicle Rental Bills =====\n");

    for (int i = 0; i < vehicles.length; i++) {

        System.out.println("Vehicle : " + names[i]);

        vehicles[i].displayDetails(days);

        System.out.println("-----");

    }

}
}

```

The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a base class 'Vehicle' with methods 'calculateRent' and 'displayDetails'. Three subclasses, 'Car', 'Bike', and 'Bus', override 'calculateRent' with different rates. The 'Main' class creates an array of these vehicles and a names array, then iterates through them to print rental details for 5 days.

```

1- class Vehicle {
2-     double calculateRent(int days) {
3-         return 0;
4-     }
5-     void displayDetails(int days) {
6-         System.out.println("Rental Days : " + days);
7-         System.out.println("Total Rent : $" + calculateRent(days));
8-     }
9- }
10- class Car extends Vehicle {
11-     @Override
12-     double calculateRent(int days) {
13-         return days * 1500;
14-     }
15- }
16- class Bike extends Vehicle {
17-     @Override
18-     double calculateRent(int days) {
19-         return days * 500;
20-     }
21- }
22- class Bus extends Vehicle {
23-     @Override
24-     double calculateRent(int days) {
25-         return days * 3000;
26-     }
27- }
28- public class Main {
29-     public static void main(String[] args) {
30-         Vehicle[] vehicles = {
31-             new Car(),
32-             new Bike(),
33-             new Bus()
34-         };
35-         String[] names = {
36-             "Car",

```

The Output window shows the following results:

```

===== Vehicle Rental Bills =====

Vehicle : Car
Rental Days : 5
Total Rent : $7500.0
-----
Vehicle : Bike
Rental Days : 5
Total Rent : $2500.0
-----
Vehicle : Bus
Rental Days : 5
Total Rent : $15000.0
-----

=== Code Execution Successful ===

```

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

```
import java.util.Scanner;

class BankAccount {

    private int accountNumber;

    private String holderName;

    private double balance;

    BankAccount(int accountNumber, String holderName, double balance) {

        this.accountNumber = accountNumber;

        this.holderName = holderName;

        this.balance = balance;

    }

    public void deposit(double amount) {

        if (amount > 0) {

            balance += amount;

            System.out.println("Deposit Successful.");

        } else {

            System.out.println("Invalid Deposit Amount.");

        }

    }

    public void withdraw(double amount) {

        if (amount <= 0) {

            System.out.println("Invalid Withdrawal Amount.");

        } else if (amount > balance) {

            System.out.println("Insufficient Balance.");

        } else {

            balance -= amount;

        }

    }

}
```

```

        System.out.println("Withdrawal Successful.");
    }
}

public void displayAccount() {
    System.out.println("\n----- Account Details ----- ");
    System.out.println("Account Number : " + accountNumber);
    System.out.println("Holder Name   : " + holderName);
    System.out.println("Balance       : $" + balance);
}
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Account Number: ");
        int accNo = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Account Holder Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Initial Balance: ");
        double balance = sc.nextDouble();
        BankAccount account = new BankAccount(accNo, name, balance);
        int choice;
        do {
            System.out.println("\n===== Bank Account Menu =====");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Balance Inquiry");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");

```

```
choice = sc.nextInt();

switch (choice) {

    case 1:

        System.out.print("Enter Deposit Amount: ");

        double deposit = sc.nextDouble();

        account.deposit(deposit);

        break;

    case 2:

        System.out.print("Enter Withdrawal Amount: ");

        double withdraw = sc.nextDouble();

        account.withdraw(withdraw);

        break;

    case 3:

        account.displayAccount();

        break;

    case 4:

        System.out.println("Thank You!");

        break;

    default:

        System.out.println("Invalid Choice.");

}

} while (choice != 4);

sc.close();

}
```

```

Main.java
1: import java.util.Scanner;
2: class BankAccount {
3:     private int accountNumber;
4:     private String holderName;
5:     private double balance;
6:     BankAccount(int accountNumber, String holderName, double balance) {
7:         this.accountNumber = accountNumber;
8:         this.holderName = holderName;
9:         this.balance = balance;
10:    }
11:    public void deposit(double amount) {
12:        if (amount > 0) {
13:            balance += amount;
14:            System.out.println("Deposit Successful.");
15:        } else {
16:            System.out.println("Invalid Deposit Amount.");
17:        }
18:    }
19:    public void withdraw(double amount) {
20:        if (amount <= 0) {
21:            System.out.println("Invalid Withdrawal Amount.");
22:        } else if (amount > balance) {
23:            System.out.println("Insufficient Balance.");
24:        } else {
25:            balance -= amount;
26:            System.out.println("Withdrawal Successful.");
27:        }
28:    }
29:    public void displayAccount() {
30:        System.out.println("\n----- Account Details -----");
31:        System.out.println("Account Number : " + accountNumber);
32:        System.out.println("Holder Name : " + holderName);
33:        System.out.println("Balance : $" + balance);
34:    }
35: }
36: public class Main {
37:     public static void main(String[] args) {
38:         Scanner sc = new Scanner(System.in);
39:         System.out.print("Enter Account Number: ");
40:         int accNo = sc.nextInt();
41:         sc.nextLine();
42:         System.out.print("Enter Account Holder Name: ");
43:         String name = sc.nextLine();
44:         System.out.print("Enter Initial Balance: ");
45:         double balance = sc.nextDouble();
46:         BankAccount account = new BankAccount(accNo, name, balance);
47:         int choice;
    }
}

```

```

Output
Enter Account Number: 2001
Enter Account Holder Name: JAYANTHI
Enter Initial Balance: 5000

===== Bank Account Menu =====
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter your choice: 2
Enter Withdrawal Amount: 3500
Withdrawal Successful.

===== Bank Account Menu =====
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter your choice: 1
Enter Deposit Amount: 2500
Deposit Successful.

===== Bank Account Menu =====
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter your choice: 3

----- Account Details -----
Account Number : 2001
Holder Name : JAYANTHI
Balance : $4000.0

===== Bank Account Menu =====
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter your choice: 4

Thank You!

=== Code Execution Successful ===

```

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

```
class Admission {
```

```
    void calculateFee() {
```

```
        System.out.println("\n----- Undergraduate Student ----- ");
```

```
        System.out.println("Course Fee : $50000");
```

```
    }
```

```
    void calculateFee(int pg) {
```

```
        System.out.println("\n----- Postgraduate Student ----- ");
```

```
        System.out.println("Course Fee : $70000");
```

```
    }
```

```
    void calculateFee(double scholarship) {
```

```

        double originalFee = 50000;

        double finalFee = originalFee - (originalFee * scholarship / 100);

        System.out.println("\n----- Scholarship Student -----");

        System.out.println("Original Fee      : $" + originalFee);

        System.out.println("Scholarship      : " + scholarship + "%");

        System.out.println("Fee to be Paid   : $" + finalFee);

    }

}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Admission admission = new Admission();

        int choice;

        do {

            System.out.println("\n===== University Admission System =====");

            System.out.println("1. Undergraduate Student");

            System.out.println("2. Postgraduate Student");

            System.out.println("3. Scholarship Student");

            System.out.println("4. Exit");

            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {

                case 1:

                    admission.calculateFee();

                    break;

                case 2:

                    admission.calculateFee(1);

                    break;

                case 3:

```



```

        System.out.print("Enter Scholarship Percentage: ");

        double percentage = sc.nextDouble();

        admission.calculateFee(percentge);

        break;

    case 4:

        System.out.println("Thank You!");

        break;

    default:

        System.out.println("Invalid Choice.");

    }

} while (choice != 4);

sc.close();

}

}

```

The screenshot shows a Java IDE with a file named 'Main.java'. The code defines an 'Admission' class with methods to calculate fees for Undergraduate, Postgraduate, and Scholarship students, based on a scholarship percentage. A 'Main' class uses a scanner to interact with the user, displaying a menu and processing their choices. The output window on the right shows the execution flow: it starts with a menu, the user chooses '1' (Undergraduate), then '2' (Postgraduate), then '3' (Scholarship). For the scholarship choice, the user enters '80' for the percentage. The program then calculates and displays the original fee (\$50,000), the scholarship amount (80.0%), and the final fee to be paid (\$10,000). After displaying a 'Thank You!' message, the program ends with a 'Code Execution Successful' message.

```

Main.java
1 import java.util.Scanner;
2 class Admission {
3     void calculateFee() {
4         System.out.println("\n----- Undergraduate Student -----");
5         System.out.println("Course Fee : $50000");
6     }
7     void calculateFee(int pg) {
8         System.out.println("\n----- Postgraduate Student -----");
9         System.out.println("Course Fee : $70000");
10    }
11    void calculateFee(double scholarship) {
12        double originalFee = 50000;
13        double finalFee = originalFee - (originalFee * scholarship / 100);
14        System.out.println("\n----- Scholarship Student -----");
15        System.out.println("Original Fee      : $" + originalFee);
16        System.out.println("Scholarship      : " + scholarship + "%");
17        System.out.println("Fee to be Paid   : $" + finalFee);
18    }
19 }
20 public class Main {
21     public static void main(String[] args) {
22         Scanner sc = new Scanner(System.in);
23         Admission admission = new Admission();
24         int choice;
25         do {
26             System.out.println("\n==== University Admission System =====");
27             System.out.println("1. Undergraduate Student");
28             System.out.println("2. Postgraduate Student");
29             System.out.println("3. Scholarship Student");
30             System.out.println("4. Exit");
31             System.out.print("Enter your choice: ");
32             choice = sc.nextInt();
33             switch (choice) {
34                 case 1:
35                     admission.calculateFee();
36                     break;
37                 case 2:
38                     admission.calculateFee(1);
39                     break;
40                 case 3:
41                     System.out.print("Enter Scholarship Percentage: ");
42                     double percentage = sc.nextDouble();
43                     admission.calculateFee(percentge);
44                     break;
45                 case 4:
46                     System.out.println("Thank You!");
47                     break;

```

```

Output
==== University Admission System =====
1. Undergraduate Student
2. Postgraduate Student
3. Scholarship Student
4. Exit
Enter your choice: 1

----- Undergraduate Student -----
Course Fee : $50000

==== University Admission System =====
1. Undergraduate Student
2. Postgraduate Student
3. Scholarship Student
4. Exit
Enter your choice: 2

----- Postgraduate Student -----
Course Fee : $70000

==== University Admission System =====
1. Undergraduate Student
2. Postgraduate Student
3. Scholarship Student
4. Exit
Enter your choice: 3
Enter Scholarship Percentage: 80

----- Scholarship Student -----
Original Fee      : $50000.0
Scholarship      : 80.0%
Fee to be Paid   : $10000.0

==== University Admission System =====
1. Undergraduate Student
2. Postgraduate Student
3. Scholarship Student
4. Exit
Enter your choice: 4
Thank You!

=== Code Execution Successful ===

```

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

```
import java.util.Scanner;
```

```
abstract class Shape {
```

```
    abstract double area();
```

```
}
```

```
class Circle extends Shape {
```

```
    double radius;
```

```
    Circle(double radius) {
```

```
        this.radius = radius;
```

```
    }
```

```
    @Override
```

```
    double area() {
```

```
        return 3.14 * radius * radius;
```

```
    }
```

```
}
```

```
class Rectangle extends Shape {
```

```
    double length, width;
```

```
    Rectangle(double length, double width) {
```

```
        this.length = length;
```

```
        this.width = width;
```

```
    }
```

```
    @Override
```

```
    double area() {
```

```
        return length * width;
```

```
    }
```

```
}
```

```

class Triangle extends Shape {

    double base, height;

    Triangle(double base, double height) {

        this.base = base;

        this.height = height;

    }

    @Override

    double area() {

        return 0.5 * base * height;

    }

}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Shape[] shapes = new Shape[3];

        System.out.print("Enter Radius of Circle: ");

        double radius = sc.nextDouble();

        shapes[0] = new Circle(radius);

        System.out.print("Enter Length of Rectangle: ");

        double length = sc.nextDouble();

        System.out.print("Enter Width of Rectangle: ");

        double width = sc.nextDouble();

        shapes[1] = new Rectangle(length, width);

        System.out.print("Enter Base of Triangle: ");

        double base = sc.nextDouble();

        System.out.print("Enter Height of Triangle: ");

        double height = sc.nextDouble();

        shapes[2] = new Triangle(base, height);

        System.out.println("\n===== Area of Shapes =====");
    }

}

```

```

String[] shapeNames = {"Circle", "Rectangle", "Triangle"};

for (int i = 0; i < shapes.length; i++) {

    System.out.println(shapeNames[i] + " Area = " + shapes[i].area());

}

sc.close();

}

}

```

The screenshot shows an IDE with a file named 'Main.java'. The code defines an abstract class 'Shape' with an abstract method 'area()'. It then defines three subclasses: 'Circle', 'Rectangle', and 'Triangle', each implementing the 'area()' method. The 'Circle' class uses a radius of 3.14, 'Rectangle' uses length and width, and 'Triangle' uses base and height. A 'Main' class uses a 'Scanner' to take input for each shape and prints their areas. The output window shows the following text:

```

Enter Radius of Circle: 6
Enter Length of Rectangle: 4
Enter Width of Rectangle: 2
Enter Base of Triangle: 5
Enter Height of Triangle: 5

==== Area of Shapes ====
Circle Area = 113.03999999999999
Rectangle Area = 8.0
Triangle Area = 12.5

=== Code Execution Successful ===

```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface `MedicalRecord` containing methods `addRecord()` and `displayRecord()`. Implement the interface in classes `Patient` and `Doctor`. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

```
interface MedicalRecord {

    void addRecord();

    void displayRecord();

}
```

```
class Patient implements MedicalRecord {

    int patientId;

    String patientName;
```

String disease;

Scanner sc = new Scanner(System.in);

public void addRecord() {

 System.out.print("Enter Patient ID: ");

 patientId = sc.nextInt();

 sc.nextLine();

 System.out.print("Enter Patient Name: ");

 patientName = sc.nextLine();

 System.out.print("Enter Disease: ");

 disease = sc.nextLine();

}

public void displayRecord() {

 System.out.println("\n----- Patient Record----- ");

 System.out.println("Patient ID : " + patientId);

 System.out.println("Patient Name : " + patientName);

 System.out.println("Disease : " + disease);

}

}

class Doctor implements MedicalRecord {

 int doctorId;

 String doctorName;

 String specialization;

 Scanner sc = new Scanner(System.in);

 public void addRecord() {

 System.out.print("Enter Doctor ID: ");

 doctorId = sc.nextInt();

 sc.nextLine();

 System.out.print("Enter Doctor Name: ");

 doctorName = sc.nextLine();

```

        System.out.print("Enter Specialization: ");

        specialization = sc.nextLine();

    }

    public void displayRecord() {

        System.out.println("\n----- Doctor Record----- ");

        System.out.println("Doctor ID      : " + doctorId);

        System.out.println("Doctor Name    : " + doctorName);

        System.out.println("Specialization : " + specialization);

    }

}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        MedicalRecord patient = new Patient();

        MedicalRecord doctor = new Doctor();

        int choice;

        do {

            System.out.println("\n===== Hospital Management System =====");

            System.out.println("1. Add Patient Record");

            System.out.println("2. Display Patient Record");

            System.out.println("3. Add Doctor Record");

            System.out.println("4. Display Doctor Record");

            System.out.println("5. Exit");

            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {

                case 1:

                    patient.addRecord();

                    break;

```

case 2:

```
patient.displayRecord();
```

```
break;
```

case 3:

```
doctor.addRecord();
```

```
break;
```

case 4:

```
doctor.displayRecord();
```

```
break;
```

case 5:

```
System.out.println("Thank You!");
```

```
break;
```

default:

```
System.out.println("Invalid Choice.");
```

```
}
```

```
} while (choice != 5);
```

```
sc.close();
```

```
}
```

```
}
```

```
Main.java
5 }
6 class Patient implements MedicalRecord {
7     int patientId;
8     String patientName;
9     String disease;
10    Scanner sc = new Scanner(System.in);
11    public void addRecord() {
12        System.out.print("Enter Patient ID: ");
13        patientId = sc.nextInt();
14        sc.nextLine();
15        System.out.print("Enter Patient Name: ");
16        patientName = sc.nextLine();
17        System.out.print("Enter Disease: ");
18        disease = sc.nextLine();
19    }
20    public void displayRecord() {
21        System.out.println("\n----- Patient Record -----");
22        System.out.println("Patient ID : " + patientId);
23        System.out.println("Patient Name : " + patientName);
24        System.out.println("Disease : " + disease);
25    }
26 }
27 class Doctor implements MedicalRecord {
28     int doctorId;
29     String doctorName;
30     String specialization;
31    Scanner sc = new Scanner(System.in);
32    public void addRecord() {
33        System.out.print("Enter Doctor ID: ");
34        doctorId = sc.nextInt();
35        sc.nextLine();
36        System.out.print("Enter Doctor Name: ");
37        doctorName = sc.nextLine();
38        System.out.print("Enter Specialization: ");
39        specialization = sc.nextLine();
40    }
41    public void displayRecord() {
42        System.out.println("\n----- Doctor Record -----");
43        System.out.println("Doctor ID : " + doctorId);
44        System.out.println("Doctor Name : " + doctorName);
45        System.out.println("Specialization : " + specialization);
46    }
47 }
48 public class Main {
49     public static void main(String[] args) {
50         Scanner sc = new Scanner(System.in);
51         MedicalRecord patient = new Patient();
52         MedicalRecord doctor = new Doctor();
53         int choice;
54         do {
55             System.out.println("\n===== Hospital Management System =====");
56             System.out.println("1. Add Patient Record");
57             System.out.println("2. Display Patient Record");
58             System.out.println("3. Add Doctor Record");
59             System.out.println("4. Display Doctor Record");
60             System.out.println("5. Exit");
61             System.out.print("Enter your choice: ");
62             choice = sc.nextInt();
63             switch (choice) {
64                 case 1:
65                     patient.addRecord();
66                     break;
67                 case 2:
68                     patient.displayRecord();
69                     break;
70                 case 3:
71                     doctor.addRecord();
72                     break;
73                 case 4:
74                     doctor.displayRecord();
75                     break;
76                 case 5:
77                     System.out.println("Thank You!");
78                     break;
79                 default:
80                     System.out.println("Invalid Choice.");
81             }
82         } while (choice != 5);
83         sc.close();
84     }
85 }
```

```
Output
===== Hospital Management System =====
1. Add Patient Record
2. Display Patient Record
3. Add Doctor Record
4. Display Doctor Record
5. Exit
Enter your choice: 1
Enter Patient ID: 301
Enter Patient Name: ROHINI
Enter Disease: COUGH

===== Hospital Management System =====
1. Add Patient Record
2. Display Patient Record
3. Add Doctor Record
4. Display Doctor Record
5. Exit
Enter your choice: 2

----- Patient Record -----
Patient ID : 301
Patient Name : ROHINI
Disease : COUGH

===== Hospital Management System =====
1. Add Patient Record
2. Display Patient Record
3. Add Doctor Record
4. Display Doctor Record
5. Exit
Enter your choice: 3
Enter Doctor ID: 10001
Enter Doctor Name: GOPIKA
Enter Specialization: ALL

===== Hospital Management System =====
1. Add Patient Record
2. Display Patient Record
3. Add Doctor Record
4. Display Doctor Record
5. Exit
Enter your choice: 4

----- Doctor Record -----
Doctor ID : 10001
Doctor Name : GOPIKA
Specialization : ALL

===== Hospital Management System =====
1. Add Patient Record
2. Display Patient Record
3. Add Doctor Record
4. Display Doctor Record
5. Exit
Enter your choice: 5
Thank You!

*** Code Execution Successful ***
```

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

```
class Product {

    private int productId;

    private String productName;

    private double price;

    Product(int productId, String productName, double price) {

        this.productId = productId;

        this.productName = productName;

        this.price = price;

    }

    public double getPrice() {

        return price;

    }

    public void displayProduct() {

        System.out.println("\n----- Product Details ----- ");

        System.out.println("Product ID : " + productId);

        System.out.println("Product Name : " + productName);

        System.out.println("Price      : $" + price);

    }

}

class Order {

    private Product product;

    private int quantity;
```



```

Order(Product product, int quantity) {

    this.product = product;

    this.quantity = quantity;

}

public double calculateTotal() {

    return product.getPrice() * quantity;

}

public void displayBill() {

    System.out.println("\n===== ORDER BILL =====");

    product.displayProduct();

    System.out.println("Quantity    : " + quantity);

    System.out.println("Total Amount : $" + calculateTotal());

}

}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("===== Online Shopping Order System =====");

        System.out.print("Enter Product ID: ");

        int id = sc.nextInt();

        sc.nextLine();

        System.out.print("Enter Product Name: ");

        String name = sc.nextLine();

        System.out.print("Enter Product Price: ");

        double price = sc.nextDouble();

        System.out.print("Enter Quantity: ");

        int quantity = sc.nextInt();

        Product p = new Product(id, name, price);

        Order o = new Order(p, quantity);

```

```

        o.displayBill();

        sc.close();

    }

}

```

The screenshot shows an IDE with a Java file named 'Main.java'. The code defines a 'Product' class with attributes 'productId', 'productName', and 'price', and methods 'getPrice()' and 'displayProduct()'. It also defines an 'Order' class with attributes 'product' and 'quantity', and methods 'calculateTotal()' and 'displayBill()'. A 'Main' class contains the 'main' method, which uses a 'Scanner' to take user input for product ID, name, price, and quantity, then creates a product and an order object to calculate and display the bill.

```

1- import java.util.Scanner;
2- class Product {
3-     private int productId;
4-     private String productName;
5-     private double price;
6-     Product(int productId, String productName, double price) {
7-         this.productId = productId;
8-         this.productName = productName;
9-         this.price = price;
10-    }
11-    public double getPrice() {
12-        return price;
13-    }
14-    public void displayProduct() {
15-        System.out.println("\n---- Product Details ----");
16-        System.out.println("Product ID : " + productId);
17-        System.out.println("Product Name : " + productName);
18-        System.out.println("Price : $" + price);
19-    }
20- }
21- class Order {
22-     private Product product;
23-     private int quantity;
24-     Order(Product product, int quantity) {
25-         this.product = product;
26-         this.quantity = quantity;
27-     }
28-     public double calculateTotal() {
29-         return product.getPrice() * quantity;
30-     }
31-     public void displayBill() {
32-         System.out.println("\n==== ORDER BILL =====");
33-         product.displayProduct();
34-         System.out.println("Quantity : " + quantity);
35-         System.out.println("Total Amount : $" + calculateTotal());
36-     }
37- }
38- public class Main {
39-     public static void main(String[] args) {
40-         Scanner sc = new Scanner(System.in);
41-         System.out.println("==== Online Shopping Order System =====");
42-         System.out.print("Enter Product ID: ");
43-         int id = sc.nextInt();
44-         sc.nextLine();
45-         System.out.print("Enter Product Name: ");

```

The Output window shows the following text:

```

===== Online Shopping Order System =====
Enter Product ID: 205
Enter Product Name: SHAMPOO
Enter Product Price: 110
Enter Quantity: 50

===== ORDER BILL =====

----- Product Details -----
Product ID : 205
Product Name : SHAMPOO
Price : $110.0
Quantity : 50
Total Amount : $5500.0

=== Code Execution Successful ===

```

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

```
class Student {
```

```
    private int rollNo;
```

```
    private String name;
```

```
    private int mark1, mark2, mark3;
```

```
    Student(int rollNo, String name, int mark1, int mark2, int mark3) {
```

```
        this.rollNo = rollNo;
```

```
        this.name = name;
```

```
        this.mark1 = mark1;
```

```
        this.mark2 = mark2;
```

```

        this.mark3 = mark3;

    }

    public int calculateTotal() {

        return mark1 + mark2 + mark3;

    }

    public double calculateAverage() {

        return calculateTotal() / 3.0;

    }

    public String calculateGrade() {

        double avg = calculateAverage();

        if (avg >= 90)

            return "A";

        else if (avg >= 75)

            return "B";

        else if (avg >= 60)

            return "C";

        else if (avg >= 50)

            return "D";

        else

            return "Fail";

    }

    public void display() {

        System.out.println("\n----- ");

        System.out.println("Roll Number : " + rollNo);

        System.out.println("Name      : " + name);

        System.out.println("Total Marks : " + calculateTotal());

        System.out.println("Average    : " + calculateAverage());

        System.out.println("Grade      : " + calculateGrade());

    }

```

```

public int getTotal() {

    return calculateTotal();

}

public String getName() {

    return name;

}

}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number of Students: ");

        int n = sc.nextInt();

        Student[] students = new Student[n];

        for (int i = 0; i < n; i++) {

            System.out.println("\nEnter Details of Student " + (i + 1));

            System.out.print("Roll Number: ");

            int roll = sc.nextInt();

            sc.nextLine();

            System.out.print("Name: ");

            String name = sc.nextLine();

            System.out.print("Marks in Subject 1: ");

            int m1 = sc.nextInt();

            System.out.print("Marks in Subject 2: ");

            int m2 = sc.nextInt();

            System.out.print("Marks in Subject 3: ");

            int m3 = sc.nextInt();

            students[i] = new Student(roll, name, m1, m2, m3);

        }

        System.out.println("\n===== Student Results =====");
    }
}

```

```

Student topper = students[0];

for (int i = 0; i < n; i++) {

    students[i].display();

    if (students[i].getTotal() > topper.getTotal()) {

        topper = students[i];

    }

}

System.out.println("\n===== Class Topper =====");

System.out.println("Topper Name : " + topper.getName());

System.out.println("Total Marks : " + topper.getTotal());

sc.close();

}

}

```

```

Main.java
1- import java.util.Scanner;
2- class Student {
3-     private int rollNo;
4-     private String name;
5-     private int mark1, mark2, mark3;
6-     Student(int rollNo, String name, int mark1, int mark2, int mark3) {
7-         this.rollNo = rollNo;
8-         this.name = name;
9-         this.mark1 = mark1;
10-        this.mark2 = mark2;
11-        this.mark3 = mark3;
12-    }
13-    public int calculateTotal() {
14-        return mark1 + mark2 + mark3;
15-    }
16-    public double calculateAverage() {
17-        return calculateTotal() / 3.0;
18-    }
19-    public String calculateGrade() {
20-        double avg = calculateAverage();
21-        if (avg >= 90)
22-            return "A";
23-        else if (avg >= 75)
24-            return "B";
25-        else if (avg >= 60)
26-            return "C";
27-        else if (avg >= 50)
28-            return "D";
29-        else
30-            return "Fail";
31-    }
32-    public void display() {
33-        System.out.println("\n-----");
34-        System.out.println("Roll Number : " + rollNo);
35-        System.out.println("Name : " + name);
36-        System.out.println("Total Marks : " + calculateTotal());
37-        System.out.println("Average : " + calculateAverage());
38-        System.out.println("Grade : " + calculateGrade());
39-    }
40-    public int getTotal() {
41-        return calculateTotal();
42-    }
43-    public String getName() {
44-        return name;
45-    }
46- }
47- public class Main {
48-     public static void main(String[] args) {
49-         Scanner sc = new Scanner(System.in);
50-         System.out.print("Enter Number of Students: ");
51-         int n = sc.nextInt();

```

```

Output
Enter Number of Students: 3
Enter Details of Student 1
Roll Number: 01
Name: ANU
Marks in Subject 1: 90
Marks in Subject 2: 67
Marks in Subject 3: 98
Enter Details of Student 2
Roll Number: 02
Name: BANU
Marks in Subject 1: 97
Marks in Subject 2: 88
Marks in Subject 3: 94
Enter Details of Student 3
Roll Number: 03
Name: DINESH
Marks in Subject 1: 67
Marks in Subject 2: 56
Marks in Subject 3: 87
===== Student Results =====
-----
Roll Number : 1
Name : ANU
Total Marks : 255
Average : 85.0
Grade : B
-----
Roll Number : 2
Name : BANU
Total Marks : 279
Average : 93.0
Grade : A
-----
Roll Number : 3
Name : DINESH
Total Marks : 210
Average : 70.0
Grade : C
-----
===== Class Topper =====
Topper Name : BANU
Total Marks : 279
=== Code Execution Successful ===

```

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

```
import java.util.Scanner;

class Device {
    protected String deviceName;

    Device(String deviceName) {
        this.deviceName = deviceName;
    }

    void turnOn() {
        System.out.println(deviceName + " is ON");
    }

    void turnOff() {
        System.out.println(deviceName + " is OFF");
    }

    double powerConsumption() {
        return 0;
    }

    void displayPower() {
        System.out.println("Power Consumption : " + powerConsumption() + " Watts");
    }
}

class Light extends Device {
    Light() {
        super("Light");
    }

    @Override
    double powerConsumption() {
        return 10;
    }
}

class Fan extends Device {
```

```

    Fan() {
        super("Fan");
    }

    @Override
    double powerConsumption() {
        return 75;
    }
}

class AirConditioner extends Device {
    AirConditioner() {
        super("Air Conditioner");
    }

    @Override
    double powerConsumption() {
        return 1500;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Device device;

        System.out.println("===== Smart Home Device Controller =====");
        System.out.println("1. Light");
        System.out.println("2. Fan");
        System.out.println("3. Air Conditioner");
        System.out.print("Enter your choice: ");
        int choice = sc.nextInt();
        switch (choice) {
            case 1:
                device = new Light();
                break;
            case 2:
                device = new Fan();
                break;
            case 3:

```

```

        device = new AirConditioner();

        break;

    default:

        System.out.println("Invalid Choice");

        sc.close();

        return;

    }

    System.out.println("\n===== Device Details =====");

    device.turnOn();

    device.displayPower();

    device.turnOff();

    sc.close();

}

}

```

The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a base class 'Device' and three subclasses: 'Light', 'Fan', and 'AirConditioner'. The 'Device' class has methods for turning on/off, displaying power consumption, and a protected 'deviceName' attribute. The subclasses override the 'powerConsumption()' method with values 10, 75, and 1500 respectively. The main method (not fully visible) uses a scanner to take user input and interacts with the 'Device' objects. The 'Output' pane on the right shows the execution results, including a menu, user choice '1', and the resulting device details for a Light.

```

Main.java
1- import java.util.Scanner;
2- class Device {
3-     protected String deviceName;
4-     Device(String deviceName) {
5-         this.deviceName = deviceName;
6-     }
7-     void turnOn() {
8-         System.out.println(deviceName + " is ON");
9-     }
10-    void turnOff() {
11-        System.out.println(deviceName + " is OFF");
12-    }
13-    double powerConsumption() {
14-        return 0;
15-    }
16-    void displayPower() {
17-        System.out.println("Power Consumption : " + powerConsumption() + " Watts");
18-    }
19- }
20- class Light extends Device {
21-     Light() {
22-         super("Light");
23-     }
24-
25-     @Override
26-     double powerConsumption() {
27-         return 10;
28-     }
29- }
30- class Fan extends Device {
31-     Fan() {
32-         super("Fan");
33-     }
34-
35-     @Override
36-     double powerConsumption() {
37-         return 75;
38-     }
39- }
40- class AirConditioner extends Device {
41-     AirConditioner() {
42-         super("Air Conditioner");
43-     }
44-
45-     @Override
46-     double powerConsumption() {
47-         return 1500;
48-     }
49- }

```

Output

```

===== Smart Home Device Controller =====
1. Light
2. Fan
3. Air Conditioner
Enter your choice: 1

===== Device Details =====
Light is ON
Power Consumption : 10.0 Watts
Light is OFF

=== Code Execution Successful ===

```